

Lecture 4 Review Handout

Statistics 367-1-4361

Lecture Overview

Lecture 4 expands the simple Bayesian pipeline into a full **Bayesian workflow**. The lecture expands our idea of Bayesian data analysis to include a series of tests for the validity of the workflow. These include prior predictive testing, sampling diagnostics, and posterior predictive testing. Each serves a different purpose. The lecture spends the bulk of the time on how sampling works and what sampling diagnostics to look at. It introduces specific sampling algorithms: rejection sampling, MCMC, Metropolis, HMC, and Nuts. It describes specific diagnostics including the trace plot, Rhat, the autocorrelation plot, ESS, MCSE, and divergences. Different diagnostics reflect different failure modes of the sampling algorithm. The lecture ends by returning to workflow validation: prior predictive checks, diagnostics, and posterior predictive checks each test a different failure mode.

Lecture Sections

4.1 — Bayesian workflow

This section introduces the central idea of the lecture: Bayesian analysis is not just **data** → **model** → **sample** → **posterior** → **inference**. Each stage can fail, so the workflow requires explicit checks before posterior inference should be trusted. The three major checks introduced here are model coherence, sampling validity, and model fit.

New Definitions and Terms

- **Bayesian workflow** — an iterative process for building, checking, sampling, and revising statistical models
- **Sampling validity** — whether the algorithm actually sampled from the posterior distribution
- **Model fit** — whether the fitted model can generate data resembling the observed data

- **Prior predictive sampling** — simulating parameters from the prior and data from the likelihood before fitting the model
- **Posterior predictive sampling** — simulating new data from parameter values drawn from the posterior
- **Iterative workflow** — the process of revising data procedures, priors, models, parameterizations, or sampler settings when checks fail

Figure

The full Bayesian workflow includes 3 tests

- Does the model make sense
- Is the sampling algorithm actually sampling the posterior
- Does the posterior distribution fit the data

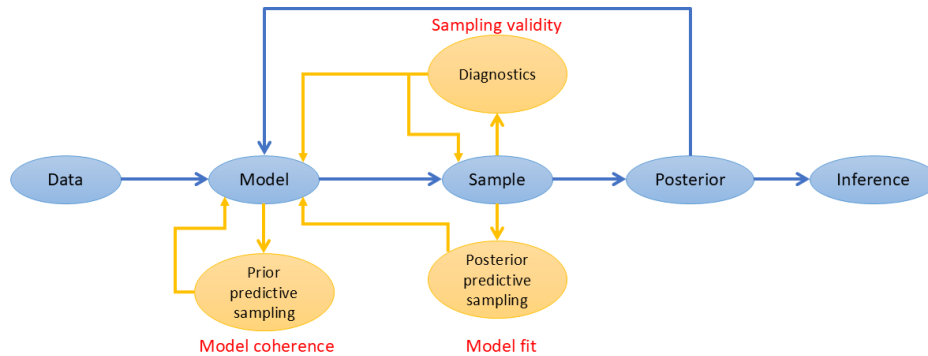


Figure 1: Slide 6 — The full Bayesian workflow includes 3 tests

4.2 — Sampling challenges

This section explains why `pm.sample()` cannot always be treated as a simple random-number generator. In conjugate models, such as the beta-binomial model, the posterior belongs to a known distribution family. But with a non-Beta prior such as a truncated normal prior on a coin probability, the posterior is only proportional to likelihood times prior, and ordinary distribution samplers are no longer enough. Rejection sampling is introduced as a simple general method for sampling a hard target distribution by sampling from an easier proposal distribution.

New Definitions and Terms

- **Truncated normal prior** — a normal prior restricted to a bounded interval, here used for $\theta \in [0,1]$
- **Rejection sampling** — sampling from an easier proposal distribution and accepting only some proposed draws
- **Proposal function** — the easy-to-sample distribution used to generate candidate draws
- **Target distribution** — the distribution the chain is designed to sample from, usually the posterior
- **Acceptance rate** — the fraction of proposed draws kept by the sampler

Formulas

$$p(\theta | y) \propto p(y | \theta)p(\theta)$$

The posterior is proportional to the likelihood times the prior. The proportional form is enough for many samplers, even when the normalizing constant is unknown.

$$Mq(x) \geq p(x)$$

In rejection sampling, the scaled proposal must cover the target distribution.

$$\text{accept if } u < \frac{p(x)}{Mq(x)}$$

A candidate draw is kept with probability equal to the target height divided by the scaled proposal height.

Figure

The advantages and challenges of rejection sampling

- Broad range of distributions
- Simple algorithm
- Samples are independent
- Finding and fitting the proposal function
- Sensitive to shape of proposal function
- Inefficient

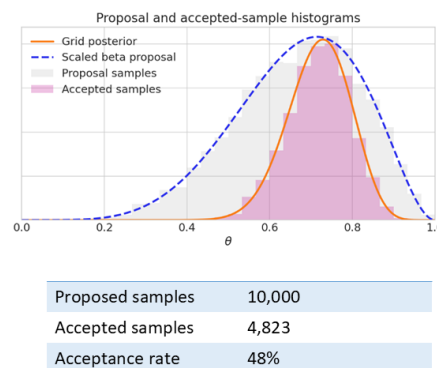


Figure 2: Slide 14 — The advantages and challenges of rejection sampling

4.3 — Monte-Carlo Markov Chain

This section introduces MCMC as a way to turn the global proposal problem of rejection sampling into a local walk through the posterior. Each new proposal is centered on the previous accepted draw, making proposals easier to construct and often easier to accept. The cost is that the draws are no longer independent, so the chain must explore the full posterior well enough before we can trust it.

New Definitions and Terms

- **MCMC** — Monte Carlo Markov Chain; a family of algorithms that generate dependent draws from a target distribution
- **Local proposal** — a proposed next draw centered near the current draw
- **Connected support** — the requirement that the sampler can move through the full region where the posterior has probability
- **Warm-up / burn-in** — early draws discarded before the chain has reached useful sampling behavior
- **Convergence** — practical evidence that the chains are exploring the same posterior distribution

Figure

MCMC turns rejection sampling to a local walk

- Use local proposals
 - Centered on the last sample
 - Easier to find
 - Easier to fit
 - Increase acceptance rate
- But samples are no longer independent

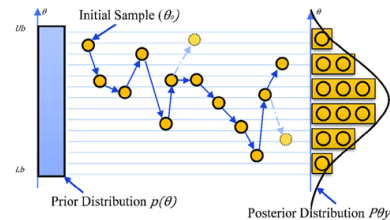


Figure 3: Slide 17 — MCMC turns rejection sampling to a local walk

4.4 — Metropolis

This section develops the Metropolis algorithm as the simplest MCMC sampler. The algorithm proposes one move at a time, always accepts moves to higher probability, and sometimes accepts moves to lower probability. The key intuition is that Metropolis is a sampler, not an optimizer: it must spend more time in high-probability regions while still exploring lower-probability regions often enough to reproduce the full target distribution.

New Definitions and Terms

- **Metropolis algorithm** — an MCMC algorithm that proposes local moves and accepts them according to a probability ratio
- **Metropolis-Hastings** — a generalization of Metropolis that corrects for asymmetric proposal distributions
- **Acceptance probability** — the probability of accepting a proposed move
- **Detailed balance** — equality of probability flow between two states in both directions under the target distribution

Formulas

$$a(x, x') = \min \left(1, \frac{p(x')}{p(x)} \right)$$

For symmetric proposals, the Metropolis acceptance probability depends only on the ratio of target probabilities.

$$\pi(x)T(x \rightarrow x') = \pi(x')T(x' \rightarrow x)$$

Detailed balance means the probability flow from x to x' equals the flow from x' to x when the chain has reached the target distribution.

$$a(x, x') = \min \left(1, \frac{p(x')q(x | x')}{p(x)q(x' | x)} \right)$$

Metropolis-Hastings adjusts the acceptance rule when the proposal is asymmetric.

Figure

Metropolis run showing
accept reject and stay

- Step 1: left, $r > 0.75$, stay
- Step 2: left, $r < 0.75$, go
- Step 3: left, $r < 0.66$, go
- Step 4: right, go
- Step 5: left, $r < 0.66$, go
- Step 6: left, $r > 0.5$, stay
- Step 7: right, go
- Step 8: left, $r < 0.66$, stay

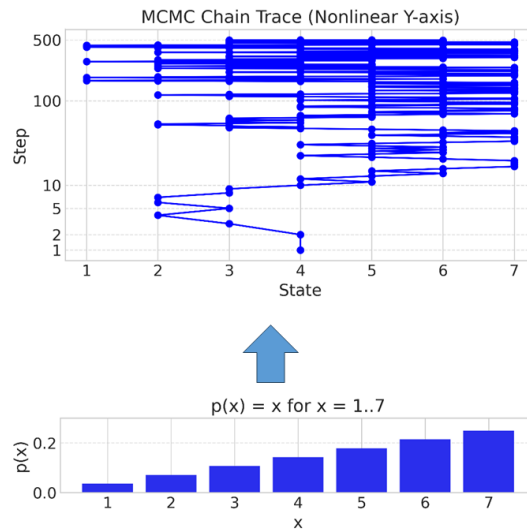


Figure 4: Slide 23 — Metropolis run showing accept reject and stay

Detailed balance: probability flow in both directions

- In the target distribution
 - Probability flows from x_1 to x_2 at the same rate it flows from x_2 to x_1
- More likely to go from 4 to 5, but more likely to be at 5 to start with

In the example ($p(\theta) \propto \theta$):

$$\begin{aligned} p(4 \rightarrow 5) &= p(5|4)p(4) = 0.5(1)\frac{4}{28} = \frac{2}{28} & p(4 \rightarrow 3) &= \frac{3}{4}0.5\frac{4}{28} = \frac{3}{56} \\ p(5 \rightarrow 4) &= p(4|5)p(5) = \frac{4}{5}0.5\frac{5}{28} = \frac{2}{28} & p(3 \rightarrow 4) &= 0.5(1)\frac{3}{28} = \frac{3}{56} \end{aligned}$$

Figure 5: Slide 28 — Detailed balance: probability flow in both directions

4.5 — Sampling diagnostics

This section separates **sampling diagnostics** from model checking. Diagnostics ask whether the sampler successfully approximated the posterior of the specified model; they do not tell us whether the model is scientifically sensible or whether it fits the observed data. The main diagnostic themes are whether chains mix with one another and whether within-chain samples carry enough independent information.

New Definitions and Terms

- **Sampling diagnostics** — checks of whether the sampling algorithm approximately sampled the posterior
- **Mixing** — the degree to which chains overlap and explore the same posterior region
- **Trace plot** — a plot of draws across iterations, used to inspect chain behavior
- **Stuck chain** — a chain that remains trapped in one region instead of exploring the posterior
- $\hat{R}$ — a convergence diagnostic comparing between-chain variation to within-chain variation
- **Autocorrelation** — dependence between nearby draws in a chain
- **ESS** — effective sample size; the number of independent draws with roughly equivalent information content

- **MCSE** — Monte Carlo standard error; uncertainty in posterior summaries caused by finite sampling

Formulas

$$\text{MCSE} \approx \frac{\text{sd}(\theta)}{\sqrt{\text{ESS}}}$$

Monte Carlo uncertainty gets smaller when the posterior is sampled more effectively.

Figure

Trace plots reveal mixing and stuck chains

```
ax = az.plot_trace(idata, var_names=['theta'], chain_props={"ls": "-"}, compact=False)
```

- - “Stuck” sampling
- - Lack of overlap

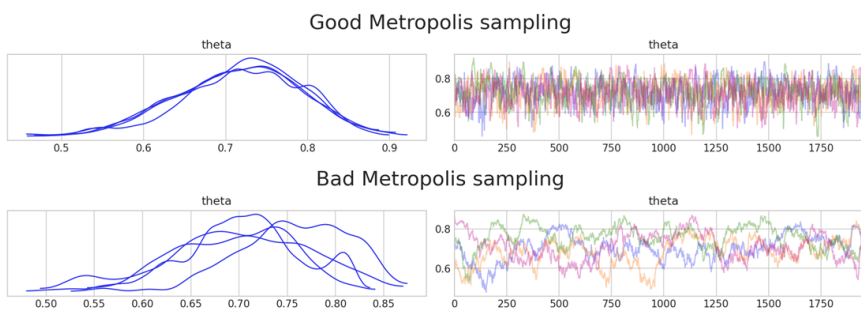


Figure 6: Slide 39 — Trace plots reveal mixing and stuck chains

Diagnostics summary

- First look
 - Trace plot
 - Look for:
 - Chain overlap
 - Consistent pattern of noise
 - Similar posteriors
- Rhat
 - Measures overlap of chains
 - Should be less than 1.1
- ESS
 - Measures within chain correlation
 - Gives information about how much information there is
 - Should be more than 500
- MCSE
 - How well are we measuring the parameter
 - Should be less than 1-2%

Figure 7: Slide 44 — Diagnostics summary

4.6 — HMC and NUTS

This section explains why simple random-walk proposals become inefficient in higher-dimensional and correlated posteriors. HMC uses gradients to build proposals that move through high-probability regions more efficiently. The key conceptual shift is that HMC is not doing gradient ascent toward the mode; in high dimensions, most probability mass is in the **typical set**, not at the point of highest density.

HMC simulates a physical system with momentum and leapfrog integration, then applies a Metropolis acceptance step. NUTS automates the difficult choice of HMC path length by stopping before the trajectory turns back on itself.

New Definitions and Terms

- **HMC** — Hamiltonian Monte Carlo; an MCMC method using gradients and simulated physical dynamics to propose efficient moves
- **Typical set** — the region where most posterior probability mass lies
- **Probability mass** — the combination of probability density and volume
- **Momentum** — the auxiliary quantity that controls HMC movement through parameter space
- **Leapfrog integration** — the numerical method used to approximate HMC trajectories

- **Target acceptance probability** — tuning parameter controlling the tradeoff between slower, more careful proposals and faster sampling
- **NUTS** — No-U-Turn Sampler; an HMC variant that automatically chooses path length
- **Maximum tree depth** — NUTS tuning parameter controlling the maximum number of trajectory steps

Figure

Probability mass combines density and volume

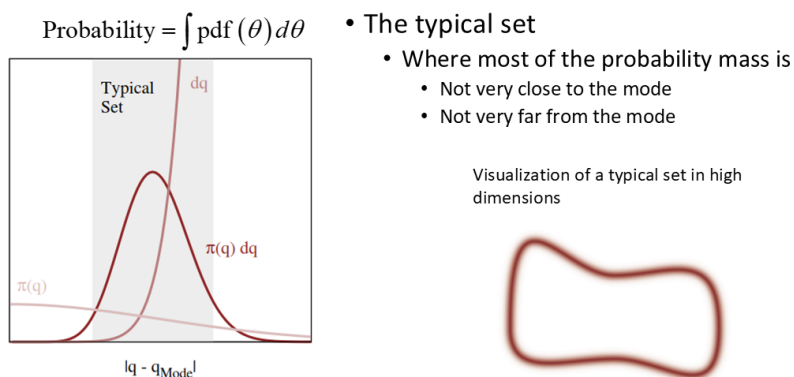


Figure 8: Slide 52 — Probability mass combines density and volume

HMC proposals stay in the typical set

- HMC moves perpendicular to the gradient
 - Like a satellite
 - Not towards the mode
 - Not away from it



Figure 9: Slide 53 — HMC proposals stay in the typical set

NUTS automates HMC path length selection

- NUTS: No U-Turn Sampling
 - A variant of HMC
 - Explores multiple possible jumps in two directions
 - Default state-of-the-art sampler in PPCs
- Parameters
 - Target acceptance probability
 - Controls step size
 - Higher gives slower but better sampling
 - Maximum tree depth
 - Controls the number of steps
 - Higher gives slower but better sampling

HMC with u turns

Figure 10: Slide 59 — NUTS automates HMC path length selection

4.7 — Divergences: HMC diagnostics

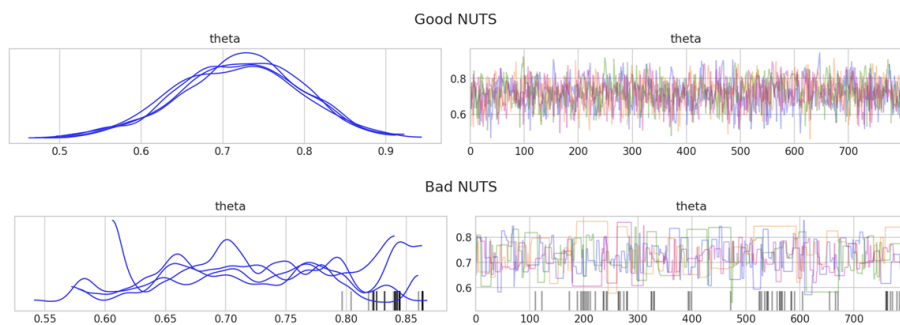
This section introduces divergences as a specific diagnostic for HMC and NUTS. A divergence occurs when the approximate physical simulation fails badly enough that the sampler detects an unreliable trajectory. In lecture terms, the satellite has either flown away from the stable orbit or crashed into the gravity well; in practical terms, posterior samples from a run with divergences cannot simply be trusted.

New Definitions and Terms

- **Divergence** — a detected failure of the HMC/NUTS trajectory simulation
- **Energy error** — change in the simulated Hamiltonian energy caused by numerical approximation failure
- **Problematic region** — an area of parameter space where the sampler repeatedly diverges
- **Reparameterization** — rewriting the model to make the posterior easier to sample

Figure

Trace plots show divergences



- This can help identify the problematic region in parameter space

Figure 11: Slide 64 — Trace plots show divergences

4.8 — PyTensor

This section explains where PyMC gets the mathematical objects needed by modern samplers. The user writes a probabilistic model in PyMC, but HMC and NUTS require numerical functions for the log probability density and its gradients. PyTensor provides this computational backend by turning symbolic model variables into probability calculations and computational graphs.

New Definitions and Terms

- **PyTensor** — PyMC’s computational backend for symbolic probability calculations and gradients
- **Symbolic variable** — a variable that represents a mathematical quantity before numerical values are supplied
- **Computational graph** — a graph representation of the operations needed to compute model probabilities
- **Log joint probability density** — the log probability of parameters and observed data together
- **Gradient of the log probability** — the derivative information used by HMC/NUTS to build efficient proposals
- **Transformed parameter** — an internal representation of a constrained parameter used by PyMC during sampling

Formulas

$$\log p(\theta, y) = \log p(\theta) + \log p(y \mid \theta)$$

The log joint probability combines the prior and likelihood terms.

$$\nabla_{\theta} \log p(\theta, y)$$

The gradient tells the sampler how log probability changes as parameter values change.

Figure

PyMC symbolic variables become probability calculations

- PyMC `basic_RVs` shows the random variables in our model
- PyTensor `dprint` show the graph of relationships

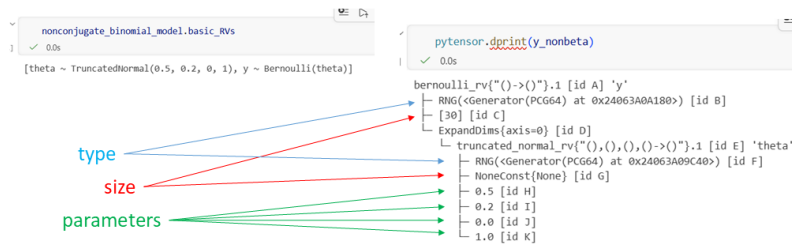


Figure 12: Slide 69 — PyMC symbolic variables become probability calculations

4.9 — Workflow validation

This final section returns to the full Bayesian workflow and clarifies what each check is testing. Prior predictive checks test whether the priors and likelihood together generate plausible data before seeing the observed dataset. Diagnostics test whether sampling succeeded. Posterior predictive checks test whether the fitted model can generate data that resemble the observed data.

The important workflow message is that failures have different meanings. A failed prior predictive check points back to the model or priors. Failed diagnostics point to the sampler, parameterization, or model complexity. A failed posterior predictive check points to model fit. Only after these checks does the posterior become a serious basis for inference.

New Definitions and Terms

- **Predictive sampling** — generating data by first drawing parameters and then drawing data conditional on those parameters

Figure

Bayesian workflow is iterative and self-correcting

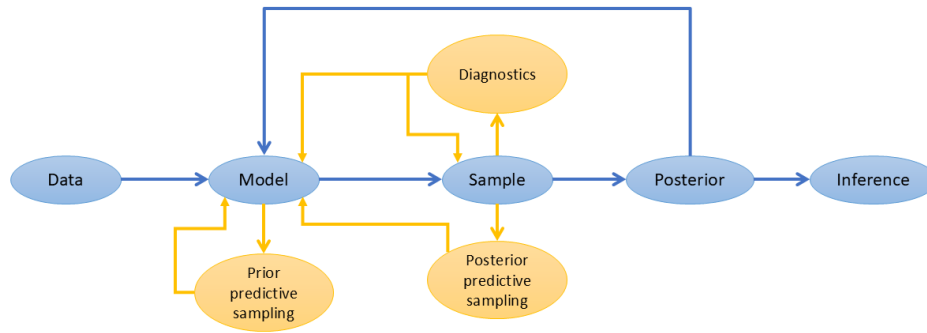


Figure 13: Slide 73 — Bayesian workflow is iterative and self-correcting

Each failure mode has different solutions

Step	Test	Solutions
Data collection	Data validation procedures	Improve methodology Develop protocols Document processes
Propose model	Generate graph Take samples	Test dimensions Debug
Sample prior predictive	Prior predictive plots	Simplify model Change priors
Sample posterior	Trace plots Rhat ESS MCSE Divergences	Improve sampling Change initialization Reparameterize model Simplify model Change priors
Sample posterior predictive	Posterior predictive plots Bayesian p values	Change model

87

Figure 14: Slide 87 — Each failure mode has different solutions